

EXAM NOTES

P3: Redirection — dup(), dup2() & I/O Redirection

Complete Beginner Guide — Coded Diagrams, Worked Examples, MCQ Answers

⚡ INSTANT REFERENCE

Function / Symbol	What it does in one sentence
<code>dup(oldfd)</code>	Copies oldfd to the LOWEST available fd number; returns the new fd
<code>dup2(oldfd, newfd)</code>	Copies oldfd to exactly newfd; closes newfd first if already open
<code>close(fd)</code>	Closes the file descriptor; frees that slot in the FD table
<code>open(path, flags)</code>	Opens a file; returns the lowest unused fd (or -1 on error)
<code>< infile</code>	Shell: redirect stdin (fd 0) to read from infile
<code>> outfile</code>	Shell: redirect stdout (fd 1) to write to outfile
<code>>> outfile</code>	Shell: redirect stdout to APPEND to outfile
<code>2> errfile</code>	Shell: redirect stderr (fd 2) to errfile
<code>&> file</code>	Shell: redirect BOTH stdout and stderr to file
<code>FD 0 / 1 / 2</code>	stdin / stdout / stderr — always open at program start

1. HOW THE FILE DESCRIPTOR TABLE WORKS

Every process owns a File Descriptor Table (FDT). It maps small integers (0, 1, 2, 3 ...) to actual open files managed by the OS.

Three-Level File System Structure

Level	What it contains	Example entry
FD Table (per process)	Integers → pointers into OS Open File Table	FD 1 → entry #7
Open File Table (OS-wide)	Mode (read/write), file position, reference count	Write mode, position=0
Inode Table (OS-wide)	Actual file name, location on disk	/tmp/out.txt

□ KEY

Think of FDs as nicknames. FD 1 is just 'stdout' — a nickname for whatever file is currently assigned to slot 1.
You redirect output by changing WHICH file slot 1 points to — not by modifying the program.
The OS always uses the lowest available fd number when you call `open()` or `dup()`.

Default State at Program Start

Default FD Table (every program starts like this)		
FD #	Name	Points to...
0	stdin	Keyboard (terminal input)
1	stdout	Terminal (screen output)
2	stderr	Terminal (error output)
3	(unused)	—
4	(unused)	—

□ TIP

When you call `open()` and fds 0, 1, 2 are all in use, the OS returns 3.
When you call `open()` again (with 3 in use), you get 4. Always the LOWEST free number.
`fileno(stdin)=0, fileno(stdout)=1, fileno(stderr)=2`

2. SHELL REDIRECTIONS — THE SYMBOLS

When you type a command in bash, you can attach `<`, `>`, `>>` etc. to redirect where input comes from or output goes. The program itself does NOT change — only its FDs are rewired.

All Redirection Symbols

Shell syntax	Meaning — what fd is rewired and how
<code>cmd < infile</code>	FD 0 (stdin) now reads from infile instead of keyboard
<code>cmd > outfile</code>	FD 1 (stdout) now writes to outfile (TRUNCATES existing content)
<code>cmd >> outfile</code>	FD 1 (stdout) appends to outfile (does NOT erase old content)
<code>cmd 2> errfile</code>	FD 2 (stderr) now writes to errfile
<code>cmd 2>> errfile</code>	FD 2 (stderr) appends to errfile
<code>cmd &> file</code>	Both FD 1 and FD 2 go to file
<code>cmd 2>&1</code>	FD 2 (stderr) goes to wherever FD 1 currently points
<code>cmd > /dev/null</code>	Discard output — /dev/null swallows everything written to it

Shell Redirection Example — sort

```
$ sort < file.txt > sorted.txt
# sort reads its stdin (fd 0) ← from file.txt
# sort writes its stdout (fd 1) → to sorted.txt
# sort's code is UNCHANGED — it still reads fd 0 and writes fd 1
# The shell rewired the FDs before launching sort
```

❏ TIP

The program does not know it has been redirected — it still reads fd 0 and writes fd 1. Redirection is transparent to the program. Only the FD table entry changes. /dev/null is a black hole — anything written there is silently discarded. Great for suppressing output.

More Shell Examples

```
$ ls > files.txt ◀ stdout → files.txt (creates/truncates)
$ ls >> files.txt ◀ stdout → files.txt (appends)
$ gcc a.c 2> errors.txt ◀ stderr → errors.txt
$ gcc a.c > out.txt 2>&1 ◀ stdout → out.txt; stderr → same place as stdout
$ ./prog < input.txt > out.txt ◀ stdin from file; stdout to file
$ ./noisy > /dev/null ◀ discard all stdout output
```

❏ EXAM

MCQ: 'cmd > out.txt' uses which fd? Answer: fd 1 (stdout).
MCQ: '2>&1' means? Answer: redirect fd 2 to wherever fd 1 currently points.
MCQ: Does 'sort < in > out' modify sort's source code? Answer: NO — only FDs change.

3. dup() AND dup2() — DUPLICATING FILE DESCRIPTORS

dup() and dup2() let you make two FD numbers point to the same underlying file. This is the core mechanism behind I/O redirection in C programs.

Signatures

```
#include <unistd.h>

int dup(int oldfd);
// Copies oldfd to the LOWEST available fd slot
// Returns: the new fd number, or -1 on error

int dup2(int oldfd, int newfd);
// Copies oldfd to EXACTLY newfd
// If newfd is already open, it is CLOSED first (atomically)
// Returns: newfd on success, or -1 on error
// If oldfd == newfd, does nothing and returns newfd
```

dup() — Goes to Lowest Available Slot

Example: FDs 0, 1, 2 are open. FD 3 is first available.

```
int newfd = dup(1); // fd 1 = stdout ◀ newfd will be 3
// Now BOTH fd 1 and fd 3 point to stdout
// Writing to fd 3 == writing to stdout
```

After dup(1) — FD 1 and FD 3 both point to stdout		
FD #	Name	Points to...
0	stdin	Terminal (keyboard)
1	stdout	Terminal (screen)
2	stderr	Terminal (screen)
3	dup'd	Terminal (screen) ◀ SAME as fd 1

TIP

dup() is useful to SAVE a file descriptor before closing it.
Save fd 1: int saved = dup(1); — later restore with: dup2(saved, 1); close(saved);

dup2() — Goes to Exact Slot (The One You Need for Redirection)

Example: FD 3 is open to a file. We want fd 1 (stdout) to point to that file instead.

```
dup2(3, 1); ◀ fd 1 now points to same file as fd 3
// dup2 automatically closed the OLD fd 1 (stdout) first
// Now printf() writes to the FILE, not the terminal
```

After dup2(3, 1) — stdout now goes to the file		
FD #	Name	Points to...
0	stdin	Terminal (keyboard)

1	stdout	myfile.txt ← REDIRECTED (was terminal)
2	stderr	Terminal (screen)
3	file	myfile.txt

⚠ WARN

dup2(oldfd, newfd) CLOSES newfd FIRST if it is already open — no need to call close(newfd) before dup2.
This is why dup2 is preferred over the two-step close(1); dup(3); — dup2 is atomic.
After dup2(3,1): fd 1 AND fd 3 both point to the file. You usually close(3) next to clean up.

dup() vs dup2() — Side by Side

Aspect	dup(oldfd)	dup2(oldfd, newfd)
Target fd	Lowest available (you don't choose)	Exactly newfd (you choose)
Closes newfd?	N/A — picks an unused slot	YES — closes newfd first if open
Atomic?	No (need close+dup separately)	YES — close+copy in one call
Returns	The new fd number	newfd (or -1 on error)
Use case	Saving a copy of a fd	Redirecting stdin/stdout/stderr

🔑 KEY

For redirecting stdout/stderr, ALWAYS use dup2() — it's safer and atomic.
Two-step method (close+dup) has a race condition: another thread could grab the fd between the two calls.

4. REDIRECTING STDOUT — COMPLETE STEP-BY-STEP

Goal: make `printf()` write to a file instead of the terminal. This is exactly what bash does when you type `./prog > outfile`.

The 3-Step Recipe (memorise this!)

- ❑ **STEPS**
 - Step 1 — `open()`: Open the target file. Get back a new fd (usually 3).
 - Step 2 — `dup2()`: Copy that fd to slot 1 (stdout). fd 1 now points to the file.
 - Step 3 — `close()`: Close the original fd (3). No longer needed — fd 1 does the job.

Full Code Example — Redirect stdout to a file

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    // STEP 1: open the output file
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644); ◀ fd = 3
    if (fd == -1) { perror("open"); return 1; }

    // STEP 2: make fd 1 (stdout) point to output.txt
    dup2(fd, 1); ◀ fd 1 now → output.txt

    // STEP 3: close the now-redundant fd 3
    close(fd); ◀ fd 3 freed; fd 1 still → output.txt

    // Now printf writes to output.txt, NOT the terminal!
    printf("This goes into output.txt\n");
    printf("So does this line.\n");

    return 0;
}
```

STEP 1 — After `open('output.txt')`

FD #	Name	Points to...
0	stdin	Terminal keyboard
1	stdout	Terminal screen
2	stderr	Terminal screen
3	file	output.txt (newly opened)

STEP 2 — After `dup2(fd, 1)` — fd 1 now points to output.txt

FD #	Name	Points to...
0	stdin	Terminal keyboard
1	stdout	output.txt ← REDIRECTED!
2	stderr	Terminal screen

3	file	output.txt (same as fd 1 for now)
---	------	-----------------------------------

STEP 3 — After close(fd) — clean, fd 3 freed		
FD #	Name	Points to...
0	stdin	Terminal keyboard
1	stdout	output.txt ✓
2	stderr	Terminal screen
3	(unused)	—

□ KEY	After Step 3: printf() → fd 1 → output.txt. Terminal shows nothing. stderr (fd 2) still goes to the terminal — error messages still appear there. close(3) does NOT affect fd 1, because dup2 made them independent copies.
-------	---

Method 1 vs Method 2 — Why dup2 Wins

```
// Method 1 - TWO steps, NOT atomic (avoid this)
close(1);          // fd 1 is now free
dup(3);            // copies fd 3 to lowest free fd - which is now 1
// Problem: between close(1) and dup(3), another thread could grab fd 1!

// Method 2 - dup2, ATOMIC (use this)
dup2(3, 1);        // closes fd 1 AND copies fd 3 to fd 1 in one atomic step
// No race condition possible
```

□ WARN	On an exam, if asked 'which method is better?' — always say dup2() because it's atomic. Method 1 (close+dup) has a race condition between the two calls in multithreaded programs.
--------	---

5. REDIRECTING I/O FOR A CHILD PROCESS (fork + exec)

This is the heart of how bash runs programs with redirections. The pattern is: fork → set up FDs in child → exec. FDs survive exec, so the new program inherits the redirected FDs.

Why This Works

- fork() copies the parent's FD table into the child
- exec() does NOT reset FDs — the new program inherits them
- So: set up FDs in child BEFORE calling exec
- The exec'd program reads/writes its fd 0/1/2 as usual — it doesn't know they're redirected

□ KEY

The golden rule: always set up FDs in the CHILD, AFTER fork(), BEFORE exec(). After exec() it is too late — you can never go back to the old code. The parent's FDs are not affected by what the child does to its FDs.

Summary of Steps in the Child (from the slides)

□ STEPS

1. Close FDs inherited from parent that the child does NOT need.
2. open() — open the file you want to redirect to/from.
3. dup2() — copy the new fd to the right standard fd (0, 1, or 2).
4. close(fd) — close the now-redundant fd from step 2.
5. exec() — replace child with the desired program.

Full Example: Run 'ls' with stdout redirected to a file

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <stdio.h>

int main() {
    pid_t pid = fork(); ◀ creates child
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        // — CHILD: set up redirection, then exec —

        // Step 2: open the output file
        int fd = open("ls_output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644); ◀ fd=3
        if (fd == -1) { perror("open"); return 1; }

        // Step 3: redirect stdout (fd 1) to the file
        dup2(fd, 1); ◀ fd 1 → ls_output.txt

        // Step 4: close the extra fd
        close(fd); ◀ fd 3 freed

        // Step 5: exec — ls inherits fd 1 → ls_output.txt
        execlp("ls", "ls", "-l", NULL); ◀ ls writes to file, not terminal
    }
}
```

```

    perror("exec"); return 1;    // only reached if exec fails
}

// — PARENT —
wait(NULL); ◀ wait for ls to finish
printf("ls output is in ls_output.txt\n");
return 0;
}

```

□ KEY

ls does NOT know its output is going to a file. It still writes to fd 1 as always. The parent's fd 1 is unaffected — parent can still print to the terminal. 'ls_output.txt' is created by the child; parent sees it after wait() returns.

Example: Redirect stderr for a child

Goal: run gcc but capture its error messages to errors.txt (parent has fd 3 open to errors.txt):

```

if (pid == 0) {
    // Child sets up stderr redirect before exec
    dup2(3, 2); ◀ fd 2 (stderr) → errors.txt
    close(3); ◀ fd 3 no longer needed
    execlp("gcc", "gcc", "a.c", NULL); ◀ gcc error msgs → errors.txt
    perror("exec"); return 1;
}

```

Child FD table after dup2(3,2) and close(3)

FD #	Name	Points to...
0	stdin	Terminal keyboard
1	stdout	Terminal screen
2	stderr	errors.txt ← REDIRECTED!
3	(unused)	— (closed)

□ TIP

After exec (gcc runs), gcc still writes error messages to fd 2. But fd 2 now points to errors.txt, so all gcc errors land in that file. Parent's fd 2 still points to the terminal — parent's stderr is unaffected.

Example: Redirect stdin for a child (reading from a file)

```

if (pid == 0) {
    int fd = open("input.txt", O_RDONLY); ◀ fd=3
    dup2(fd, 0); ◀ fd 0 (stdin) ← input.txt
    close(fd); ◀ fd 3 freed
    execlp("sort", "sort", NULL); ◀ sort reads from input.txt
    perror("exec"); return 1;
}
// Equivalent to shell command: sort < input.txt

```

Where Should You Open the File?

This is a classic exam question: 'Where and when should the file be opened?'

Option	Correct?	Reason
Before <code>fork()</code> , in parent	✓OK (but careful)	Child inherits the open fd — works, but parent and child share the file position
After <code>fork()</code> , in parent branch	✗NO	Parent's FD setup doesn't affect child (different processes now)
After <code>fork()</code> , in CHILD branch	✓BEST	Only child gets this fd; clean separation; no sharing issues
After <code>exec()</code> in child	✗IMPOSSIBLE	<code>exec()</code> replaces your code — you can't run more code after <code>exec</code> succeeds

□ EXAM

Answer to 'when should the file be opened for stdout redirect?': After `fork()`, in the child, before `exec()`.
 Answer to 'where should `dup2()` be called?': After `fork()`, in the child, before `exec()`.
 Answer to 'where should `close(newfd)` be called?': After `dup2()`, in the child, before `exec()`.

6. HOW TO TRACE REDIRECTION CODE — OUTPUT QUESTIONS

Exam questions often show code with `fork/dup2/exec` and ask: 'What is printed on screen?' or 'What is in the file?'

The 7-Step Tracing Method

Step	What to ask yourself
1. Find <code>fork()</code>	Split into parent and child columns here
2. Identify FD changes	Which <code>dup2()</code> calls happen in each branch? What does each FD now point to?
3. Track <code>printf/write</code>	For each <code>printf()</code> , check — which fd does it use? (<code>printf</code> uses fd 1)
4. Check for <code>exec()</code>	If <code>exec</code> succeeds, trace the NEW program's output with the new FDs
5. Check for <code>exit()</code>	Does child exit cleanly? Or does it fall through?
6. Check <code>wait()</code>	Does parent wait? Does the output ORDER matter?
7. Separate destinations	Which output goes to screen? Which goes to file?

Worked Example 1 — What prints to screen vs file?

```
int main() {
    int fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
    pid_t pid = fork();

    if (pid == 0) {                // CHILD
        dup2(fd, 1);              // child: stdout → out.txt ◀ fd 1 → file
        close(fd);
        printf("I am child\n"); // goes to out.txt, NOT screen ◀ → file
        exit(0);
    } else {                      // PARENT
        close(fd);
        printf("I am parent\n"); // parent fd 1 = terminal → screen ◀ → screen
        wait(NULL);
    }
    return 0;
}
```

□ KEY

Screen shows: 'I am parent'
out.txt contains: 'I am child'
Why? Child changed its OWN fd 1. Parent's fd 1 is unaffected (separate FD tables).
`close(fd)` in the parent closes the parent's copy — child already has its `dup2'd` copy.

Worked Example 2 — Redirect then exec

```

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        int fd = open("result.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644); ◀ fd=3
        dup2(fd, 1);           // stdout → result.txt ◀ fd 1 → file
        close(fd);
        execlp("echo", "echo", "hello world", NULL); ◀ echo writes to fd 1 = file
        exit(1);
    }
    wait(NULL);
    printf("done\n");           // parent stdout = terminal ◀ → screen
    return 0;
}

```

KEY

Screen shows: 'done'
 result.txt contains: 'hello world'
 echo uses fd 1 to write — but fd 1 was redirected to result.txt before exec.
 FDs survive exec(), so echo sees the redirected fd 1.

Worked Example 3 — Saving and Restoring stdout

```

int main() {
    int saved_stdout = dup(1);    // save original stdout ◀ saved=3

    int fd = open("file.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644); ◀ fd=4
    dup2(fd, 1);                 // redirect stdout to file ◀ fd 1→file
    close(fd);

    printf("to file\n");         // goes to file.txt ◀ → file

    dup2(saved_stdout, 1);       // RESTORE original stdout ◀ fd 1→terminal
    close(saved_stdout);         // saved copy no longer needed ◀ fd 3 freed

    printf("to screen\n");       // back to terminal ◀ → screen
    return 0;
}

```

KEY

Screen shows: 'to screen'
 file.txt contains: 'to file'
 Pattern: save → redirect → use → restore. Useful when you need to temporarily redirect and then undo it.

Worked Example 4 — The Missing close() Trap

```

int main() {
    int fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
    pid_t pid = fork();
    if (pid == 0) {
        dup2(fd, 1);
        // MISSING: close(fd) ◀ fd 3 is still open in child! ◀ leak!
        execlp("ls", "ls", NULL);
        exit(1);
    }
    close(fd);    // parent closes fd 3
    wait(NULL);
    // out.txt may NOT be fully written — child still holds fd 3 open
}

```

```
    return 0;  
}
```

❏ **WARN**

Always close(fd) after dup2(fd, target) in the child.
Leaving extra fds open causes fd leaks — the OS limits how many fds a process can have.
Also: a file pipe's read-end won't get EOF if a write-end fd is accidentally left open.

7. COMMON PATTERNS — COPY-PASTE TEMPLATES

Pattern A — Redirect stdout of THIS process

```
int fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
dup2(fd, 1);           // stdout → out.txt
close(fd);
// All subsequent printf() goes to out.txt
```

Pattern B — Redirect stdin of THIS process

```
int fd = open("input.txt", O_RDONLY);
dup2(fd, 0);           // stdin ← input.txt
close(fd);
// All subsequent scanf()/gets()/read(fd=0) reads from input.txt
```

Pattern C — Redirect stderr of THIS process

```
int fd = open("errors.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
dup2(fd, 2);           // stderr → errors.txt
close(fd);
// All subsequent fprintf(stderr,...) / perror() goes to errors.txt
```

Pattern D — Redirect stdout for a CHILD program (fork+exec)

```
pid_t pid = fork();
if (pid == 0) {
    int fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
    dup2(fd, 1);        // child's stdout → out.txt
    close(fd);
    execlp("ls", "ls", NULL);
    perror("exec"); exit(1);
}
wait(NULL);
```

Pattern E — Redirect both stdin AND stdout

```
if (pid == 0) {
    int in = open("input.txt", O_RDONLY);
    int out = open("output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
    dup2(in, 0);        // stdin ← input.txt
    dup2(out, 1);       // stdout → output.txt
    close(in);
    close(out);
    execlp("sort", "sort", NULL);
    // equivalent to: sort < input.txt > output.txt
    perror("exec"); exit(1);
}
```

Pattern F — Save and Restore a fd

```
// Save original stdout
int saved = dup(1);                // saved = 3

// Redirect to file temporarily
int fd = open("temp.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
dup2(fd, 1);
close(fd);
printf("this goes to temp.txt\n");

// Restore original stdout
dup2(saved, 1);
close(saved);
printf("back to terminal\n");
```

8. MCQ QUICK-ANSWER TABLE

Question	Answer
What does <code>dup(oldfd)</code> return?	The new fd number (lowest available), or -1 on error
What does <code>dup2(oldfd, newfd)</code> return?	<code>newfd</code> on success, -1 on error
<code>dup2(3, 1)</code> — what does it do to old fd 1?	Closes it first, then copies fd 3 into slot 1
After <code>dup2(3,1)</code> , do fd 1 and fd 3 share position?	Yes — they point to the same open file description
Why is <code>dup2</code> better than <code>close(1)+dup(3)</code> ?	<code>dup2</code> is atomic — no race condition between close and copy
Do FDs survive <code>fork()</code> ?	Yes — child inherits copies of all parent FDs
Do FDs survive <code>exec()</code> ?	Yes — new program inherits all open FDs
When should you redirect FDs for a child?	After <code>fork()</code> , in the child, BEFORE <code>exec()</code>
Can you redirect after <code>exec()</code> ?	NO — <code>exec</code> replaces your code; no code runs after it
What does <code>close(fd)</code> after <code>dup2</code> do?	Frees the extra fd slot; the <code>dup2</code> 'd target fd still works
<code>dup2(fd, 0)</code> redirects which standard stream?	stdin (standard input)
<code>dup2(fd, 1)</code> redirects which standard stream?	stdout (standard output)
<code>dup2(fd, 2)</code> redirects which standard stream?	stderr (standard error)
Shell '> file' uses which fd?	fd 1 (stdout)
Shell '< file' uses which fd?	fd 0 (stdin)
Shell '2> file' uses which fd?	fd 2 (stderr)
What is <code>/dev/null</code> ?	A special file — anything written to it is discarded
Does redirecting child FDs affect parent FDs?	NO — parent and child have separate FD tables after <code>fork()</code>
What happens if you forget <code>exit()</code> in child?	Child falls through and runs parent code — double execution bug
What if <code>dup2()</code> 's <code>oldfd</code> is invalid (= -1)?	<code>dup2</code> returns -1 and sets <code>errno</code> — always check <code>open()</code> returned valid fd first

9. FINAL ONE-PAGE CHEATSHEET

Concept / Function	Quick fact — exam ready
FD table	Per-process map: integers → open files. OS fills lowest unused slot.
FD 0 / 1 / 2	stdin / stdout / stderr — always open at start
open() returns	Lowest unused fd number, or -1 on error
dup(oldfd)	Copy oldfd → lowest free slot. Returns new fd.
dup2(oldfd, newfd)	Copy oldfd → exactly newfd. Closes newfd first. Atomic.
close(fd)	Free that fd slot. Other dup'd copies still work.
Redirect stdout	open() → dup2(fd,1) → close(fd)
Redirect stdin	open() → dup2(fd,0) → close(fd)
Redirect stderr	open() → dup2(fd,2) → close(fd)
Fork + redirect + exec	fork → [child] open → dup2 → close → exec
FDs after fork()	Inherited. Shared underlying file position!
FDs after exec()	All survive. New program sees same FDs.
Redirect in child WHEN?	After fork(), before exec() — never after exec
Shell < infile	fd 0 reads from infile
Shell > outfile	fd 1 writes to outfile (truncates)
Shell >> outfile	fd 1 appends to outfile
Shell 2> errfile	fd 2 writes to errfile
Shell 2>&1	fd 2 → wherever fd 1 currently points
Shell &> file	Both fd 1 and fd 2 → file
/dev/null	Black hole — discards all data written to it
dup2 vs close+dup	dup2 is ATOMIC and preferred. close+dup has race condition.
Close after dup2	Always close(fd) after dup2(fd, target) — avoid fd leaks
Missing exit() in child	Child runs parent code — double output bug

Good luck on your exam! ☐ Remember: open → dup2 → close → exec